

Exact Acceleration of a Scanning Unknot Recognizer

Sparse cancellation, finite-ring obstructions,
and connected-sum compression

Computational research and implementation report
Companion to the accelerated `fastunknot` source distribution

18 September 2026

Abstract

We improve the Python unknot recognizer in the supplied `Knots.zip`, rather than replacing its complete algorithm with an incomplete invariant test. The implementation combines exact one-sided modular Alexander and Jones obstructions, a faster descending-diagram test, an asymptotically faster version of the original greedy scan ordering, sparse unit cancellation in the dotted cobordism category, and visible connected-sum factorization. Every successful verdict retains a complete mathematical justification; resource exhaustion is reported separately. In process-isolated tests, complete recognition of the Conway and Kinoshita–Terasaka examples improves by factors of about 33 and 24. An unfactored 36-crossing backend workload that did not finish in a separate 120-second baseline run completes in 2.81 seconds, with reduced rank 2,949. For connected sums of fixed bounded-size diagrams, factorwise rank computation is polynomial, while the original explicit minimal-complex representation can require exponentially many objects even at constant boundary width. A 132-crossing example with reduced rank 33^{12} takes 0.18 seconds and a peak of 246 objects in any factor scan. We prove the transformations, describe the validation and measured regressions, and audit the relationship to Lackenby’s hierarchy approach. *No general quasi-polynomial bound is established for the delivered implementation.* Its general worst-case bound remains $2^{O(n)}$; the asymptotic gains are in preprocessing and specified decomposable families, in addition to the measured practical gains.

Contents

1	Scope, input, and the baseline	2
2	Attempting the quasi-polynomial route	3
2.1	The quantitative bridge that an implementation would need	3
3	The exact portfolio and its decision invariant	4
4	Faster preprocessing without changing its meaning	4
4.1	Linear descending-start detection	4
4.2	The same greedy order in near-linear time	5
5	Exact finite-ring obstructions	5
5.1	Evaluating an Alexander minor directly	5
5.2	A writhe convention that must not be guessed	6
5.3	Loop-inclusive normalized bracket	6
5.4	Matching-state dynamic programming	7

6	Accelerating exact sparse cancellation	7
6.1	What the scanner stores	7
6.2	Every unit is self-inverse in characteristic two	7
6.3	Fill-aware Gaussian pivots	8
6.4	Caching without hidden algebraic mutation	8
7	Connected sums and avoiding explicit homology multiplicities	9
7.1	Finding visible splitting curves	9
7.2	Why unreduced ranks can be halved degree by degree	9
7.3	Tensor products and homological convolution	10
7.4	An actual family-level asymptotic improvement	10
8	Completeness and general worst-case complexity	11
9	Executed experiments	12
9.1	Measurement protocol	12
9.2	Complete recognition	12
9.3	Unfactored homology backend	12
9.4	Factored ranks and preprocessing	13
9.5	Validation results and what they do not prove	13
10	Reproduction, interfaces, and limitations	14
A	Auditable workload and source map	15
B	Rebuilding the report and checking results	15

1 Scope, input, and the baseline

The target is the `fast/` implementation in the user’s archive. Its unmodified Python package, tests, and examples are retained under `baseline/`; the replacement package is `fastunknot/`. The result is executable with the Python standard library alone. The report is about this implementation and these experiments, not a claim to have developed a new mathematical unknot-detection theorem.

An input is a classical, one-component knot diagram with n crossings. A planar diagram (PD) row is $[a, b, c, d]$ in counterclockwise order, with the under-strand joining a to c and the over-strand joining b to d . Edge labels occur twice and are normalized to $0, \dots, 2n - 1$. The empty PD denotes *one* circle, not the empty link. Constructors also accept braid closures and rectangular (grid) diagrams. They check the strand-component count and the Euler characteristic of the specified rotation system. An abstractly planar graph with a nonspherical rotation system is not accepted as a classical diagram. Low-level scan routines retain the baseline precondition that their PD is valid; applications should use the validated `Diagram` constructors.

The original recognition pipeline performs crossing-decreasing Reidemeister I and II moves, tests for a descending traversal, computes an exact Alexander polynomial, and finally computes Khovanov homology over $\mathbb{F}_2$ by scanning. The last step is based on local complexes, delooping, and Gaussian cancellation in the dotted-cobordism framework of Bar–Natan [3]. The implementation forgets quantum degree and retains unnormalized cube degree. For a knot, the total unreduced $\mathbb{F}_2$ rank is twice the reduced rank. Reduced rank one detects the unknot; Section 8 explains the coefficient-field issue.

The supplied baseline is already a genuine complete recognizer when resource limits are absent. The improvement must therefore preserve exactness, not merely recognize a larger collection of easy examples. Our changes fall into three categories: polynomial preprocessing improvements, exact rejection filters, and changes to the computation and representation of the complete homological fallback. The raw-backend and complete-recognition timings are deliberately reported separately.

2 Attempting the quasi-polynomial route

The attached talk announces a quasi-polynomial algorithm. Its final flowchart contains hierarchical multi-surfaces, cutting, Cheeger-region simplification, Haken's lemma, weak reduction, and multi-surface simplification. The preceding slides use generalized Heegaard splittings and explain that absence of a Cheeger region yields logarithmic hierarchy length [2]. These are three-dimensional topological operations, not alternate names for Khovanov matrix pivots.

The attached July 2026 paper is a related hierarchy algorithm. Section 9 gives an iteration bound $L(g+1)^L$ and explicitly distinguishes it from a running-time estimate. Section 10 then refines the algorithm using interior parallelity bundles and gives a linear bound on L in the initial q -complexity [1]. Thus it would be inaccurate either to call the Section 9 formula a proved bit-complexity bound, or to say that the paper never controls hierarchy length.

2.1 The quantitative bridge that an implementation would need

Let $B(n)$ bound the number of bits in a maintained topological state and let $P(B)$ bound the cost of an elementary operation. Suppose a hierarchy-based implementation established

$$T(n) \leq n^a L(g+1)^L P(B(n)), \quad L \leq c \log(n+2), \quad g \leq (n+2)^d, \quad (1)$$

with $B(n) \leq n^{O(1)}$ and polynomial P . Taking logarithms then gives

$$\log T(n) = O(\log n) + O(\log n) O(\log n) = O((\log n)^2),$$

and hence $T(n) = n^{O(\log n)}$. This elementary calculation identifies what would actually have to be measured and proved. Replacing the logarithmic bound on L by a linear bound in n gives only $L(g+1)^L = \exp(O(n \log n))$ under the same polynomial bound on g . Furthermore, a small *number* of transitions does not prevent a transition from expanding an exponentially long normal surface.

A viable implementation contract would therefore need a compressed state containing the manifold's handle incidences, boundary pattern, surface coordinates, hierarchy references, and Heegaard data, together with actual algorithms for the transformations above. Each transformation must preserve the represented topology, maintain explicit bit-size bounds, and either decrease a proved potential or make progress in a bounded construction loop. Merely creating stubs named after the flowchart would not meet that contract.

The supplied scanning objects do not encode this information. In particular, their boundary matching is a two-dimensional smoothing interface; it is not a normal surface in a knot exterior. I did not obtain implementations and proofs of these three-dimensional contracts in this work. The delivered executable therefore does not call a putative quasi-polynomial subroutine or advertise such a guarantee. Instead, the following sections give completed improvements with explicit correctness arguments and executed tests. This is a conclusion about the present implementation, not a claim that the hierarchy program is impossible or that the announced theorem is false.

3 The exact portfolio and its decision invariant

The accelerated recognizer follows this sequence:

1. Apply the original validated decreasing R1/R2 moves; an empty diagram is UNKNOT.
2. Find a descending traversal by cyclic intervals; success gives UNKNOT.
3. Split visible connected sums. A nontrivial factor gives KNOTTED; all trivial factors give UNKNOT.
4. Apply modular Alexander necessary conditions; a mismatch gives KNOTTED.
5. Evaluate a normalized bracket modulo a fixed prime; a mismatch gives KNOTTED.
6. Compute the original exact Alexander polynomial; a nonunit gives KNOTTED.
7. Compute reduced $\mathbb{F}_2$ Khovanov rank with accelerated scanning; rank one gives UNKNOT and any other rank gives KNOTTED.

An inconclusive filter proceeds to the next stage. Exceeding the optional Jones state cap also proceeds to the next stage; exceeding the overall user time or Khovanov-object budget produces UNKNOWN. No conclusion is inferred from an exhausted search, a modular match, or the failure to find a decreasing move.

The default Jones state cap is 20,000. Modular Alexander uses the fixed prime $p = 1,000,000,007$, verified in the tests by trial division through its square root. Neither numerical roundoff nor a probabilistic identity test enters a verdict. Although a modular evaluation can miss a nontrivial polynomial, that can only cause more work, not a false UNKNOT answer.

4 Faster preprocessing without changing its meaning

4.1 Linear descending-start detection

Choose one orientation and write the $2n$ incoming darts encountered around the knot as a cyclic sequence. For crossing c , let o_c and u_c be the indices of its over-visit and under-visit. Starting the traversal at index s encounters c under first exactly when

$$s \in (o_c, u_c] \quad \text{in cyclic order.} \quad (2)$$

A descending start therefore lies outside the union of these forbidden intervals.

Proposition 4.1. *A difference-array computation finds a descending start, or certifies that none exists, in $O(n)$ combinatorial operations and $O(n)$ storage.*

Proof. Split a wrapping cyclic interval into at most two ordinary intervals. For an interval $[l, r]$, add one at l and subtract one at $r + 1$. Prefix sums count forbidden intervals covering each candidate start. A zero count is precisely a start that encounters every crossing over first, by (2). Perform the same calculation in the opposite orientation. If d is an incoming dart, the opposite incoming dart is the port two positions away at the same crossing; reversing the traversal and applying this port involution gives the other orientation. Thus the two linear scans cover the same $4n$ possible starting darts examined separately by the baseline. $\square$

The original implementation has an $O(n^2)$ worst-case upper bound because it restarts a traversal for every candidate. Some inputs fail almost immediately at each candidate, so their actual baseline cost can already be linear. The 501-crossing microbenchmark exhibits essentially no practical gain; this does not contradict the improved worst-case preprocessing bound.

4.2 The same greedy order in near-linear time

At any partial scan, let s_i be the number of edges joining an unprocessed crossing i to the processed set, and let ℓ_i be its number of self-loops. The original greedy score, after removing quantities common to all candidates, orders crossings lexicographically by

$$(s_i, \ell_i, -i).$$

Indeed, if the current boundary has size b , processing i changes it to $b+4-2s_i-2\ell_i$. The tie-breaking rule is unchanged.

Store $(-s_i, -\ell_i, i)$ in a min-heap. After processing a crossing, only its at most four incident non-loop neighbours can acquire additional shared edges. Insert their new scores and discard stale heap entries lazily. The total number of insertions is $O(n)$ and heap operations take $O(\log n)$ each. This yields the exact same order for a specified starting crossing as the original $O(n^2)$ scan over all remaining candidates.

The default tries a bounded number of starts. The inherited spacing rule can try somewhat more than 12 starts, but fewer than 24; thus the default order selection is $O(n \log n)$, versus $O(n^2)$ previously. With every crossing used as a start, the bounds become $O(n^2 \log n)$ and $O(n^3)$. These bounds count operations on normalized labels; bit costs and Python allocation constants are additional implementation details. Five hundred randomized comparisons check equality of the old and new orders at up to three starts each.

5 Exact finite-ring obstructions

5.1 Evaluating an Alexander minor directly

Use the same Wirtinger/Fox rows as the original package. Deleting one row and one column yields a polynomial determinant $D(t)$ representing the Alexander polynomial up to a unit $\pm t^k$. Rather than construct polynomial coefficient lists, evaluate each sparse row at $t = a \in \mathbb{F}_p$ first and perform ordinary Gaussian elimination over $\mathbb{F}_p$. This costs $O(n^3)$ field operations; p is fixed, so all filter coefficients have bounded bit length.

Proposition 5.1. *For a knot with unit Alexander polynomial, the following necessary conditions hold whenever $a \neq 0$ in $\mathbb{F}_p$:*

$$D(-1) \in \{1, -1\}, \quad D(a)D(a^{-1}) = 1.$$

Consequently, violating either condition proves nontriviality.

Proof. For a unit polynomial, $D(t) = \varepsilon t^k$ with $\varepsilon \in \{1, -1\}$. Substitution gives $D(-1) = \varepsilon(-1)^k$ and $D(a)D(a^{-1}) = \varepsilon^2 a^k a^{-k} = 1$. Evaluation is a ring homomorphism, so computing the determinant after evaluation gives the same residue as evaluating the determinant. The unknot has unit Alexander polynomial, establishing the contrapositive test. $\square$

The implementation tries $t = -1$ first. If inconclusive, it tries $a = 2$ and a^{-1} . The reciprocal product avoids guessing the unknown sign and power of t . It also catches some nontrivial Alexander polynomials with determinant one. It does not catch the Conway or Kinoshita–Terasaka examples, whose Alexander polynomial is identically one. The exact polynomial stage remains available after the modular and Jones stages.

5.2 A writhe convention that must not be guessed

The original public `signs()` routine used the incoming over-port alone. That value serves its fixed-port Fox-row convention, but is not the oriented crossing sign on an arbitrary valid PD. Let $u_i \in \{0, 2\}$ and $o_i \in \{1, 3\}$ be the incoming under- and over-ports. The sign compatible with this package's bracket convention is

$$\sigma_i = \begin{cases} 1 & o_i = 3, \\ -1 & o_i = 1 \end{cases}, \quad \begin{cases} 1 & u_i = 0, \\ -1 & u_i = 2. \end{cases}, \quad w(D) = \sum_i \sigma_i. \quad (3)$$

A half-turn of a PD row changes both factors, leaving the product invariant. Reversing the knot orientation likewise changes both incoming ports and leaves σ_i invariant. These two requirements immediately expose the one-port problem. In the inherited braid-to-PD convention, the single generator [1] has $w = -1$; callers must not impose an unrelated sign convention.

The old one-port values are retained as `relation_signs()`, and the Alexander code explicitly calls that routine. Its a -port is treated as the incoming port in the chosen relation, whether or not it is globally incoming. Simply replacing those relation signs with (3) would introduce a new error. The original recognition decisions did not use Jones normalization, so this finding is not evidence that the baseline's Alexander verdicts were incorrect. It is a public writhe bug and a prerequisite for a safe new filter.

5.3 Loop-inclusive normalized bracket

Fix a commutative ring and a unit A . Write $\delta = -A^2 - A^{-2}$. For a nonempty diagram define

$$B_D(A) = \sum_{s \in \{0,1\}^n} A^{n-2|s|} \delta^{c(s)}, \quad F_D(A) = (-A^3)^{-w(D)} B_D(A), \quad (4)$$

where $c(s)$ counts all closed smoothing circles. This is the loop-inclusive form of the bracket state model [7]; in particular $B_U = F_U = \delta$, rather than 1. The empty PD is handled as the one-circle special case.

The smoothing connecting $(a, b), (c, d)$ has coefficient A , and the smoothing connecting $(a, d), (b, c)$ has coefficient A^{-1} . The local R1 calculation multiplies B by $-A^3$ or $-A^{-3}$, exactly cancelled by the writhe factor under (3). R2 follows from the local relation $e^2 = \delta e$:

$$(A 1 + A^{-1} e)(A^{-1} 1 + A e) = 1 + (A^2 + A^{-2} + \delta) e = 1.$$

Expanding the two three-crossing products, using the adjacent smoothing relations $e_i e_{i+1} e_i = e_i$, gives R3. Thus F_D is a knot invariant with unknot value δ . These polynomial identities remain valid after specialization in any ring where A is invertible.

Proposition 5.2. *A modular value $F_D(A) \neq \delta$ proves that D is knotted. Equality is inconclusive, including when the chosen specialization makes $\delta = 0$.*

This test never assumes that the full Jones polynomial detects the unknot. It also never assumes that one evaluation determines the polynomial. For example, the trefoil at $A = 1$ passes this filter; the tests check that this collision is not interpreted as an unknot certificate.

5.4 Matching-state dynamic programming

A partial smoothing is represented by its matching of exposed edge endpoints. For each matching, store a scalar coefficient. Attaching the next crossing has two choices. Multiply by A or A^{-1} and by δ^c for the newly closed circles, then add coefficients of equal resulting matchings. Induction on the processed crossings proves that each stored coefficient is the sum of exactly the partial states it represents. At the end only the empty matching remains, with coefficient $B_D(A)$.

If the boundary has w endpoints, an unconditional upper bound on the number of possible matching states is

$$(w-1)!! = \frac{w!}{2^{w/2}(w/2)!} = 2^{O(w \log w)}.$$

A Catalan bound requires additional disk-boundary/noncrossing structure, which we do not assume for an arbitrary scan prefix. The state count is also at most the number of processed resolutions. With the fixed state cap, the optional filter has bounded-state polynomial overhead; without a cap, no general polynomial bound in n is asserted. A cap hit discards the partial evaluation and falls through to the complete recognizer.

6 Accelerating exact sparse cancellation

6.1 What the scanner stores

A scanning object consists of a boundary matching and a cube degree. Closed circles are delooped into choices of the two labels $1, x$ in $\mathbb{F}_2[x]/(x^2)$. Between matchings a, b , the morphism representation is a set of square-free dot monomials on the circles obtained by gluing a to the reflected b . Addition is symmetric difference. Composition is the exact two-dimensional Frobenius-algebra calculation, not a numerical approximation.

The geometry routines of the supplied implementation are preserved. Most new work removes repeated computation and chooses a different valid elimination order. Local tensoring, delooping, and cancellation are the algebraic basis of the fast scanning method [3]. Forgetting quantum grading is sufficient for total-rank detection, but these routines do not output integral torsion or a bigraded polynomial.

6.2 Every unit is self-inverse in characteristic two

For a matching with r arcs, its endomorphism ring in the implemented normal form is

$$R_r = \mathbb{F}_2[x_1, \dots, x_r]/(x_1^2, \dots, x_r^2).$$

A morphism is a unit precisely when its constant term is one.

Lemma 6.1. *If $f = 1 + \nu \in R_r$ and ν has zero constant term, then $\nu^2 = 0$ and $f^{-1} = f$.*

Proof. Write ν as a sum of nonconstant square-free monomials. In characteristic two the cross terms in its square cancel in pairs. The square of each remaining monomial contains x_i^2 for at least one i and is zero. Therefore $(1 + \nu)^2 = 1 + 2\nu + \nu^2 = 1$. $\square$

The baseline computed a geometric series for the inverse. The replacement returns a fresh copy of f . It is not merely an early truncation chosen from observed data. The same proof explains a direct local-ring composition routine: two dot monomials multiply to zero if they overlap and to their union otherwise; equal output monomials cancel by symmetric difference. Identity and zero compositions are also recognized before invoking geometry.

6.3 Fill-aware Gaussian pivots

Suppose an invertible differential entry $\phi : b \rightarrow c$ is selected. For surviving entries $\delta : a \rightarrow c$ and $\gamma : b \rightarrow f$, cancellation changes

$$d_{af} \longleftarrow d_{af} + \gamma\phi^{-1}\delta. \quad (5)$$

The sign is plus because the coefficient field is $\mathbb{F}_2$.

Lemma 6.2. *Removing b, c and updating by (5) preserves the chain-homotopy type and hence the homology rank.*

Proof. In the relevant two degrees, isolate the invertible block ϕ in the matrix of the differential. Invertible row and column operations clear its row and column, leaving the Schur complement on the other objects. The cleared block is the two-term contractible complex $b \xrightarrow{\phi} c$. The equation $d^2 = 0$ makes the adjacent differential blocks compatible with these changes of basis. Deleting the contractible summand gives exactly (5). The argument works in the additive category because the only required inverse is that of ϕ . $\square$

The number of possible new entries from this cancellation is bounded by

$$\mu(b, c) = (|\text{inc}(c)| - 1)(|\text{out}(b)| - 1).$$

The new implementation keeps unit entries in a lazy heap ordered by this Markowitz-style fill estimate, with deterministic object-ID ties. It recomputes a popped score and reinserts it when stale. Scores of other heap entries can still be stale, so the implementation does *not* promise a globally minimum current score. What matters for correctness is that every performed pivot is genuinely invertible. Every unit present initially is enqueued, and every changed entry that becomes a unit is enqueued. Thus changing the queue policy cannot stop elimination with an unexamined surviving unit.

There is no theorem here that this heuristic improves every input. Cancellation order can change transient fill dramatically, but it can also introduce heap overhead with little benefit. The experiments measure the complete package of kernel changes, not a separately randomized causal attribution to each change.

6.4 Caching without hidden algebraic mutation

Composition is bilinear. We cache the composition of individual input monomials and sum the immutable cached answers. Crossing-tensor entries are also cached with immutable morphism values; callers receive fresh mutable containers. The geometry caches use 16,384 entries each, the monomial composition cache 32,768, and the crossing-entry cache 8,192. Cache eviction only forces recomputation and has no mathematical effect.

Entry-count bounds are not byte bounds: one morphism can itself be large. The internal geometry cache objects must be treated as read-only. The tests exercise mutation of returned morphism results to check that a caller cannot corrupt a cached composition through the public arithmetic wrapper. The `clear_caches()` function supports cold experiments and long-lived applications. These caches are performance aids, not persistent mathematical assumptions.

7 Connected sums and avoiding explicit homology multiplicities

7.1 Finding visible splitting curves

In the planar dual of the knot projection, two parallel edges joining distinct faces form a simple two-edge cycle. Its geometric realization crosses exactly two edges of the original diagram. The Jordan curve separates the sphere into two disks, and closing the strand segment on each side produces diagrams D_1, D_2 with $D = D_1 \# D_2$.

The implementation computes dart-to-face incidences and groups primal edges by the unordered pair of their incident faces. For a candidate pair it deletes the two primal edges and runs a graph traversal. It checks that the partition is nontrivial and that each deleted edge has exactly one endpoint on each side. It then closes each side by identifying its two cut-edge labels, normalizes the PD, and revalidates spherical one-component structure. A trace stores the local PD and the chosen edges and crossing partition. Applying the procedure recursively yields visible diagram factors, not necessarily prime knots.

For a valid spherical projection, a dual two-cycle gives the cut directly; the primal checks are conservative safeguards. Building the dual and checking one cut is linear graph work. Repeated splitting has at most $n-1$ cuts and polynomial overhead, including revalidation and trace construction. A conservative $O(n^3)$ operation bound covers the Python implementation without assuming optimal disjoint-set or balanced splitting behavior. A diagram of a composite knot can hide every such splitting curve; this procedure does not solve general prime decomposition.

7.2 Why unreduced ranks can be halved degree by degree

The characteristic-two splitting is standard in Khovanov theory [5, 6]. Here is an algebraic proof in the conventions needed by the program. On a resolution with r circles, write its vector space as $A^{\otimes r}$, $A = \mathbb{F}_2[x]/(x^2)$. Fix a basepoint on the knot, let X be multiplication by x on its circle, and let

$$\nu = \sum_{j=1}^r \partial_{x_j}, \quad \partial_x(1) = 0, \quad \partial_x(x) = 1.$$

Both commute with the differential. For X this is the marked Frobenius-module relation. For ν it follows by checking

$$\partial m = m(\partial \otimes 1 + 1 \otimes \partial), \quad (\partial \otimes 1 + 1 \otimes \partial)\Delta = \Delta\partial$$

on the basis $1, x$, where

$$m(x, x) = 0, \quad m(1, x) = m(x, 1) = x, \quad m(1, 1) = 1, \quad \Delta(1) = 1 \otimes x + x \otimes 1, \quad \Delta(x) = x \otimes x.$$

Also $X^2 = 0$ and $\nu X + X\nu = 1$.

Proposition 7.1. *After forgetting quantum grading, the unreduced complex splits into two isomorphic copies of the reduced complex in the same cube degrees. Consequently every unreduced homological rank is even.*

Proof. Take the reduced complex to be $\text{im } X$. The identity $\nu X + X\nu = 1$ decomposes every chain element into a member of $\text{im } X$ and a member of $\nu \text{im } X$. If $y = \nu z \in \text{im } X$ with $z \in \text{im } X$, then $0 = Xy = X\nu z = z$, hence $y = 0$. Thus $C = \text{im } X \oplus \nu \text{im } X$. The maps ν and X restrict to mutually inverse chain maps between these summands. Their quantum shifts are irrelevant here; neither changes cube degree. Taking homology proves the claim. $\square$

7.3 Tensor products and homological convolution

For a marked knot diagram define the reduced rank polynomial

$$R_D(z) = \sum_h \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}^h(D; \mathbb{F}_2) z^h,$$

using the package's unnormalized cube degree h .

Proposition 7.2. *For visibly joined diagrams,*

$$R_{D_1 \# D_2}(z) = R_{D_1}(z) R_{D_2}(z).$$

In particular their total reduced ranks multiply.

Proof. A resolution of the joined diagram is a pair of resolutions of the factors. Their marked circles become a single marked circle. Label that joined circle by x in the reduced complex, and retain all labels on the other circles. This gives a basis bijection with the tensor product of the two reduced complexes. It respects the local merge and split maps: a crossing change lies in one factor, and its marked and unmarked label rules are unchanged by the joining. Thus the differential is $d_1 \otimes 1 + 1 \otimes d_2$ over $\mathbb{F}_2$, with cube degrees adding. Over a field a finite complex splits as its homology with zero differential plus contractible two-term summands. Tensoring shows that only the product of the homology summands contributes to homology, which is exactly the stated coefficient convolution. $\square$

The program scans each factor, divides its per-degree unreduced ranks by two, convolves the reduced polynomials, and doubles the final coefficients. It never constructs one Python object for every generator of the joined homology. For recognition it can stop at the first nontrivial factor; all factors must be trivial to return `UNKNOT`. The cap on live objects applies separately to the scans, not to the potentially enormous integer rank returned at the end.

7.4 An actual family-level asymptotic improvement

Theorem 7.3. *Fix a bounded-size knot diagram D with reduced rank $r > 1$. On explicitly joined diagrams $D^{\#k}$ that expose their two-edge splitting curves, the new factorwise rank algorithm runs in polynomial time in k . An algorithm which materializes the final unreduced minimal complex must use at least $2r^k$ objects. There are such families of bounded scan width.*

Proof. The splitting step and its validations are polynomial in $n = k|D|$. Each factor scan has constant size and cost. Each reduced factor polynomial has fixed degree; after j factors the product has $O(j)$ coefficients, each with $O(j)$ bits. Sequential multiplication by a fixed-degree factor is polynomial in k even with elementary integer arithmetic. Proposition 7.2 gives total reduced rank r^k , and Proposition 7.1 gives unreduced rank $2r^k$. A closed minimal complex with zero differential has one object for each of these dimensions. Finally, scanning the fixed factors successively keeps only a constant-size factor boundary and the constant number of joining strands, giving bounded width independently of k . $\square$

This is a representation improvement for specified families, not a general polynomial unknot algorithm. It also refutes a width-only bound for the *multiplicities* of the explicit Khovanov complex. A small number of boundary matching types does not bound how many copies of each type survive.

For a particularly relevant family, use the supplied 11-crossing Conway diagram C . Independent dense computation gives reduced rank 33 and exact Alexander polynomial one. The chosen splice

creates no new decreasing R1/R2 move; the supplied diagram itself has none. Its iterated sums are nontrivial, so no descending traversal exists. Their Alexander polynomial is still one. The original recognition pipeline therefore reaches its explicit homology backend, whose final size is $2 \cdot 33^k$. The new factorwise recognizer handles this family in polynomial time. This conclusion does not rely on a lucky Jones specialization of the whole sum; factoring precedes the filters and a fixed nontrivial factor is enough to decide the sum.

8 Completeness and general worst-case complexity

Theorem 8.1. *On a validated classical knot diagram, with no user resource limits, the accelerated recognizer terminates and returns **UNKNOT** precisely for the unknot. With limits, every completed verdict is sound; **UNKNOWN** is not a verdict.*

Proof. The original local reductions preserve knot type and a descending diagram is an unknot. Splitting curves preserve connected-sum structure. The Alexander and Jones rejection steps are sound by Propositions 5.1 and 5.2. The scan builds the same finite local complex as the baseline, and its changes preserve the represented morphisms and homology by Lemmas 6.1 and 6.2. Factoring preserves ranks by Proposition 7.2.

Kronheimer–Mrowka prove that reduced rational Khovanov rank one detects the unknot [4]. If the reduced $\mathbb{F}_2$ rank is one, the universal coefficient theorem bounds the reduced rational rank above by one. That rational homology is nonzero: its graded Euler characteristic is the normalized Jones polynomial, whose value at 1 for a knot is 1. Hence the rational rank is exactly one and the knot is trivial. Conversely the unknot’s reduced $\mathbb{F}_2$ complex has rank one. This justifies using $\mathbb{F}_2$ rank in the recognizer. Products of positive reduced ranks equal one exactly when every factor rank is one, which also justifies the factorwise recognition rule.

Every scan has finitely many crossings and objects; every cancellation deletes two objects. The optional filter cap skips a finite stage rather than aborting the algorithm. Thus the unlimited process terminates. Limited executions only stop early with **UNKNOWN**, without changing any algebraic test. $\square$

The implementation admits a conservative $2^{O(n)}$ upper bound on normalized inputs, not a quasi-polynomial one. Attaching a crossing chooses two smoothings and creates at most two new closed circles, so prior objects generate at most eight times as many objects before cancellation. After n steps there are at most 8^n such objects. A morphism has at most $2^{O(n)}$ dot monomials; a composition, even with naive expansion, has $2^{O(n)}$ cost. There are at most exponentially many cancellations and polynomially many pairs of incident entries per cancellation in the exponentially bounded object set. Their product is still $2^{O(n)}$. The number of generated heap entries obeys the same type of bound. Coefficient and label bit costs only contribute polynomial factors to these exponential estimates.

The Alexander stages use polynomially bounded degree and coefficient bit length, and factorization only splits the crossing set. Thus neither worsens this bound. The proof is deliberately loose; it is not a prediction of typical running time. Caches, Markowitz pivots, and greedy order selection have no proved general quasi-polynomial performance guarantee. Nor do the tests establish a new complexity lower bound for all possible homology algorithms: the lower bound in Theorem 7.3 concerns an explicit-output representation.

9 Executed experiments

9.1 Measurement protocol

The saved measurements use CPython 3.13.5, Linux x86-64, and an Intel Xeon Platinum 8370C at the reported 2.80 GHz nominal frequency. The environment exposed five logical CPUs. Each trial starts a fresh subprocess with `PYTHONHASHSEED=0`, imports exactly one implementation, validates the same PD, and then starts the kernel timer. Trials run serially. Startup, imports, and input validation are excluded from successful kernel timings; external subprocess caps include them. The machine is a shared execution environment, so small variations and sub-millisecond differences should not be overread.

`results/benchmarks.json` contains all 21 cases, exact generated input PDs, per-trial measurements, returned answers, caps, and environment data. Ordinary examples and the 256-crossing ordering use three-trial medians. The 36-crossing raw scan, large factored examples other than $C^{\#2}$, 1,024-crossing ordering, and descending microbenchmark use one trial. A first timed-out baseline trial is recorded as censored; it is not repeatedly rerun to manufacture a median. The table generator reads this JSON directly.

9.2 Complete recognition

Workload	n	Baseline (s)	New (s)	Ratio
Trefoil	3	0.000188	0.000203	0.93
Hard unknot (8)	8	0.003776	0.003626	1.04
Conway	11	0.092830	0.002823	32.89
Kinoshita–Terasaka	11	0.082476	0.003428	24.06
$T(3, 5)$	10	0.000971	0.000591	1.64
Unknot braid (40)	40	0.014090	0.011755	1.20
Recorded workload (36)	36	0.014231	0.003462	4.11

The ratio is baseline time divided by new time; values below one are regressions. The Conway and Kinoshita–Terasaka cases reach the original homology backend but are rejected by the new exact Jones evaluation. Their modular Alexander checks correctly remain inconclusive. The hard eight-crossing unknot still needs the complete fallback. The trefoil is slightly slower because the new early-stage overhead is material at a fraction of a millisecond.

The 36-crossing recorded timeout is not a difficult complete-recognition instance. Both complete pipelines reject it cheaply. The large improvement in the following subsection concerns an explicitly requested raw homology computation without the recognition filters or reductions.

9.3 Unfactored homology backend

Diagram	n	Baseline (s)	New (s)	Reduced rank
Hard unknot (8)	8	0.005712	0.004923	1
Conway	11	0.086681	0.027837	33
Kinoshita–Terasaka	11	0.099573	0.033056	33
$T(3, 5)$	10	0.022543	0.021728	7
Unknot braid (40)	40	0.019398	0.007964	1
Recorded workload (36)	36	timeout	2.810384	2949

The timed-out entry here is a 12-second external wall cap. A separate, earlier baseline run allowed 120 seconds of kernel time and stopped at 120.143533 seconds without a result; its exact record

and reproduction script are included. That observation is not a 120-second paired median and is not used to assign a numeric ratio to the censored table cell. The same workload completed in the accelerated raw scan in 2.810384 seconds. Its peak was 17,694 objects before elimination and 5,898 after elimination, with maximum boundary 8 and 16,884 cancellations. The final unreduced rank is 5,898, hence reduced rank 2,949.

An additional scan with d^2 checks completed in 3.822706 seconds with the same rank. An attempted original-backend computation using input crossing order instead of the default order exhausted its 12-second kernel budget. Thus the large rank is supported by internal differential checks and the extensive small-input comparisons below, but this work does *not* claim that the original backend independently completed that 36-crossing calculation.

The results also show why a single speedup multiplier is inappropriate. The $T(3, 5)$ kernel barely changes, whereas the large cancellation workload changes dramatically. Cold-cache small examples, not only warm repeated calls, are included. The original and new “compositions” counters have different bookkeeping conventions, so those raw counters should not be interpreted as paired operation-count ratios.

9.4 Factored ranks and preprocessing

Diagram	n	Baseline	New (s)	Reduced rank	Peak objects
$C\#^2$	22	5 s cap	0.035691	33^2	246
$C\#^3$	33	5 s cap	0.077945	33^3	246
$C\#^6$	66	5 s cap	0.096359	33^6	246
$C\#^{12}$	132	5 s cap	0.176065	33^{12}	246
$3_1\#^{10}$	30	5 s cap	0.011318	3^{10}	18

The two-factor Conway new time is a three-trial median; the other rows are single trials. All corresponding baseline first trials hit a 5-second external wall cap. The listed peak is the maximum number of live objects before elimination in *any individual factor scan*, not a process memory measure and not the returned rank. For twelve Conway factors,

$$33^{12} = 1,667,889,514,952,984,961.$$

The program returns the full unnormalized homological rank polynomial as well as this total. The final integer has modest bit length; the original explicit homology representation would have astronomical dimension. The practical speedup is consequently explained by an exact change of representation, not by discarding homology.

Workload	n	Baseline (s)	New (s)	Ratio
Greedy order, 256	256	0.483933	0.010180	47.54
Greedy order, 1,024	1024	8.117443	0.088788	91.43
Descending, 501	501	0.001404	0.001405	1.00

The ordering gains agree with the asymptotic change from quadratic to near-linear work per start. The essentially unchanged descending timing is reported rather than suppressed. As stressed earlier, a better worst-case bound for a subroutine need not make every tested input faster.

9.5 Validation results and what they do not prove

The final suite consists of 30 passing test groups. Its main independent and regression checks are as follows.

- 120 random one-component braid diagrams with at most eight crossings are compared to an independent dense reduced cube calculation, both with factoring and with the unfactored new scanner. Fifty are also compared degree by degree with the preserved original backend; 24 run intermediate d^2 checks.
- 120 diagrams are evaluated by an independent complete bracket state sum in each of three rings: the large prime field, $\mathbb{F}_{101}$, and $\mathbb{Z}/15\mathbb{Z}$. These 360 checks do not use the scanning glue routine. Additional tests cover mirrors, PD half-turns, crossing permutations, Reidemeister reductions, braid relations, and Markov stabilizations.
- 500 random diagrams compare descending-start existence and the greedy order with the original implementation. Returned descending darts are replayed. All 128 units on three arcs are checked, as are 200 general morphism compositions, cache-result mutation, modular Alexander minors, invalid budgets, Jones cap fall-through, and connected-sum degrees and traces.

The unmodified original 19 test groups also pass in a separate process. Independent dense computations on the 11-crossing Conway and Kinoshita–Terasaka diagrams give reduced rank 33 in both cases; these larger reference runs are recorded separately. The tests are deterministic, with seeds in their source. They provide substantial evidence against implementation mistakes but do not constitute a formal proof of the Python program. In particular, no Lean formalization or independently checkable succinct homology certificate has been produced.

10 Reproduction, interfaces, and limitations

From the distribution root, no installation is needed:

```
python -m fastunknot recognize examples/conway.json
python -m fastunknot recognize examples/hard_unknot_8.json
python -m fastunknot khovanov examples/baseline_timeout36.json --no-factor
python -m fastunknot khovanov examples/conway_sum_12.json
python -m unittest discover -s tests -v
python -m unittest discover -s baseline/tests -v
python tools/benchmark.py --hard-timeout 12
```

The optional installation command is `python -m pip install ..`. Runtime dependencies are limited to the standard library. The code targets Python 3.10 or later, but the delivered experiments tested only the stated 3.13.5 environment. The PDF requires an ordinary LaTeX installation; its generated tables and source are included.

`recognize` returns `UNKNOT`, `KNOTTED`, or `UNKNOWN` and records the selected method. To disable both Alexander stages use `use_alexander=False`; Jones is controlled separately by `use_jones=False`. The CLI counterparts are `-no-alexander` and `-no-jones`. A standalone Jones call explicitly labels equality `INCONCLUSIVE`. Khovanov `order=` is validated as a permutation and disables factoring, so an explicit order is never silently replaced by factor-local orders. The raw scan can also be selected with `factor=False` or `-no-factor`.

Time checks are cooperative. They now cover more loops, including the expensive optional d^2 diagnostics, but cannot interrupt a single long Python arithmetic or geometry operation. A hard service deadline must use an external process limit. The object cap is not a global memory cap. Likewise, entry-bounded caches do not impose a fixed byte ceiling. Numerical evidence and replay traces are helpful audit information, not a substitute for a separately implemented certificate verifier.

The primary remaining performance limitation is irreducible intermediate homology multiplicity on diagrams without useful visible splits. Better pivot heuristics, compiled kernels, or additional one-sided invariants could improve constants, but do not by themselves supply the missing logarithmic hierarchy-depth and compressed-state bounds of Section 2. A general quasi-polynomial successor should implement and verify those three-dimensional contracts, rather than relabel the present scanner.

A Auditable workload and source map

The archived raw-backend timeout example is the closure of the following braid on five strands:

```
[-4,-3,4,4,-3,2,1,1,-1,-1,3,-4,-1,3,2,2,-2,-1,
-3,2,3,-3,-4,-4,-3,3,1,3,-2,-4,-4,3,-3,3,-4,-2]
```

The full final unnormalized unreduced degree counts are

h	10	11	12	13	14	15	16	17	18
dim	8	40	112	230	386	552	698	784	792
h	19	20	21	22	23	24	25	26	27
dim	722	588	428	278	158	78	32	10	2

and sum to 5,898. Degree shifts are not normalized by crossing signs; the total rank is unaffected by this choice.

The central implementation locations are `scan.py` (local algebra, sparse cancellation, order selection, factored rank wrapper), `filters.py` (modular determinant and bracket DP), `decompose.py` (dual cuts and factor traces), `simplify.py` (linear descending test), `diagram.py` (separate oriented and relation signs), and `recognize.py` (sound portfolio control flow). The original code remains available for differential inspection. The patch changes only the original package’s source files; the standalone archive also provides the additional validation and benchmark artifacts.

B Rebuilding the report and checking results

```
python tools/report_tables.py
cd paper
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

`results/unit-tests.txt` and `results/baseline-unit-tests.txt` contain the executed suite outputs; `results/independent-large-checks.json` contains the larger independent reference runs and the checked 36-crossing scan. The supplementary 120-second baseline observation is in `results/baseline-hard-120s.json`. An interrupted benchmark can be resumed with `-resume -max-cases 2`; completed measurements are retained. Successful timings are not comparable to external wall caps without accounting for the different timing scopes. No checksum files, third-party font files, or copies of the cited research papers are needed to use this distribution.

References

- [1] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, 25 July 2026, especially Sections 9–10. <https://arxiv.org/abs/2607.23350>.
- [2] M. Lackenby, *Unknot recognition in quasi-polynomial time*, talk slides, 109 physical PDF pages; supplied in the input archive.
<https://people.maths.ox.ac.uk/lackenby/quasipolynomial-talk.pdf>.
- [3] D. Bar–Natan, *Fast Khovanov homology computations*, J. Knot Theory Ramifications **16** (2007), 243–255; arXiv:math/0606318. <https://arxiv.org/abs/math/0606318>.
- [4] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, Publ. Math. Inst. Hautes Études Sci. **113** (2011), 97–208; arXiv:1005.4346.
<https://arxiv.org/abs/1005.4346>.
- [5] A. Shumakovitch, *Torsion of the Khovanov homology*, arXiv:math/0405474.
<https://arxiv.org/abs/math/0405474>.
- [6] Y. Wigderson, *The Bar–Natan theory splits*, arXiv:1508.05982.
<https://arxiv.org/abs/1508.05982>.
- [7] L. H. Kauffman, *State models and the Jones polynomial*, Topology **26** (1987), 395–407.
[https://doi.org/10.1016/0040-9383\(87\)90009-7](https://doi.org/10.1016/0040-9383(87)90009-7).